

BCC Binary-Junction Node Splitting in ParaDiS

ParaDiS-Extended Development Notes

August 2026

Abstract

This note documents the implementation of three-arm BCC binary-junction splitting in the serial and parallel ParaDiS topology paths. The port follows the ExaDiS/OpenDiS implementation pinned at commit [351607fa0f04aa34d29ee253438edde994473fd6](#), with adaptations for ParaDiS node constraints, remeshing, and its serialized cross-domain operation list. It also describes the supplied one-step regression fixture and the topology expected after a successful split.

Contents

1	Scope and reference implementation	2
2	Binary-junction recognition	2
2.1	Topological and Burgers-vector tests	2
2.2	Five-degree planarity test	2
3	Split construction and acceptance	3
3.1	Allowed arm set	3
3.2	Trial mobility and geometry	3
3.3	Energetic and kinematic acceptance	3
4	Serial topology path	3
5	Parallel topology path	4
6	Corner preservation and remeshing	4
7	Cross-domain constraint propagation	4
8	Control parameters	5
9	Implementation map	5
10	Regression fixture	6
10.1	Input geometry	6
10.2	Commands	6
10.3	Expected result	6
10.4	Validation performed	6
11	Compatibility boundaries	7

1 Scope and reference implementation

The change extends the existing multi-node separation machinery, which previously considered nodes with at least four arms, to one physically specific degree-three case: a non-planar BCC binary junction. Both topology selectors use the same classifier, arm-set restriction, mobility test, acceptance threshold, split geometry, and corner-node semantics.

The source behavior was compared against the pinned upstream topology code:

https://github.com/llnl/exadis/tree/351607fa0f04aa34d29ee253438edde994473fd6/src/topology_types

The algorithm is ported rather than copied mechanically. ParaDiS has a different ownership model and communicates topology changes through a packed remote-operation stream, so explicit propagation of the new node constraint is required here.

2 Binary-junction recognition

2.1 Topological and Burgers-vector tests

The shared function `BCC_binary_junction_node(Home_t*, Node_t*, real8[3], int*)` rejects a node unless it has exactly three arms. For arm i , the minimum-image line direction is

$$\mathbf{t}_i = \frac{\text{ZImage}(\mathbf{x}_i - \mathbf{x}_0)}{\|\text{ZImage}(\mathbf{x}_i - \mathbf{x}_0)\|}. \quad (1)$$

Degenerate arms and missing neighbors cannot form a valid candidate. An arm is classified as the junction arm when

$$\|\mathbf{b}_i\| > 1 + 10^{-12}. \quad (2)$$

The node is a binary junction only when exactly one arm passes this test and the remaining two are non-junction arms (called glide arms by the topology implementation). This magnitude test does not independently validate a glissile Burgers-vector family. The classifier returns the junction arm index and its unit line direction $\mathbf{t}_j$.

Candidate construction adds the following guards:

- the material must be BCC and `split3node` must be enabled;
- planar junctions are excluded;
- surface and pinned nodes are excluded; and
- the existing short-segment filter must pass.

2.2 Five-degree planarity test

For each glide arm, the parent-plane normal is formed as

$$\mathbf{n}_i = \mathbf{b}_i \times \mathbf{t}_i. \quad (3)$$

An arm is treated as edge-bearing when $\mathbf{n}_i \cdot \mathbf{n}_i > 10^{-2}$. With $\theta_{\text{tol}} = 5^\circ$, the junction is classified as planar as follows:

- For two edge-bearing parents, normalize $\mathbf{d} = \mathbf{n}_1 \times \mathbf{n}_2$ and require $|\mathbf{d} \cdot \mathbf{t}_j| - 1| < 1 - \cos \theta_{\text{tol}}$.
- For one edge-bearing parent, require $|\mathbf{n}_1 \cdot \mathbf{t}_j| < \sin \theta_{\text{tol}}$.
- If both parents are screw-like, the junction is planar by convention.

Only a recognized junction for which this planarity result is false proceeds to split evaluation.

3 Split construction and acceptance

3.1 Allowed arm set

`SMNGetArmSets()` now supports degree-three nodes. It produces the three possible two-arm selections, while preserving the normal combinatorial enumeration for larger nodes. For a binary junction, evaluation retains only the selection containing the two glide arms; the junction arm must remain on the other trial node. The split therefore produces one degree-three node and one degree-two node connected by the newly created segment.

3.2 Trial mobility and geometry

The trial split initially creates two coincident nodes and reuses the saved arm forces. Mobility is evaluated for both nodes, the existing junction force/velocity adjustment is applied, and the velocity of the degree-two trial node is set to zero. This is the binary-junction boundary condition used by the reference implementation. Mobility return codes are combined so an error from either trial node rejects the candidate.

The candidate nodes are then separated along the accepted trial direction. The required total separation is

$$d_{\text{split}} = 2r_{\text{ann}} + 10^{-12}. \quad (4)$$

If both nodes move, each moves by half this distance; otherwise the mobile node moves by the full distance. Normalization in the opposite-velocity case is guarded against a zero norm, and the existing glide-plane rule is retained. The evaluated trial positions are saved and reused for the final three-arm split instead of being recomputed by the general `CalcJunctionLen()` path. The small positive offset prevents the new connector from lying exactly on the immediate remesh threshold.

3.3 Energetic and kinematic acceptance

Let $\mathbf{f}_k$ and $\mathbf{v}_k$ be the force and velocity of trial node k . The split power is evaluated as

$$P_s = \mathbf{f}_1 \cdot \mathbf{v}_1 + \mathbf{f}_2 \cdot \mathbf{v}_2 - v_{\text{noise}} (\|\mathbf{f}_1\| + \|\mathbf{f}_2\|), \quad (5)$$

where

$$v_{\text{noise}} = \text{splitMultiNodeAlpha} \frac{\text{rTol}}{\text{deltaTT}}. \quad (6)$$

The relative velocity must point along the proposed separation direction. The binary-junction candidate is accepted only if it improves upon the best power by more than the numerical tolerance and also satisfies $P_s > 1$. The latter threshold is specific to degree-three candidates and prevents marginal splits from being selected.

4 Serial topology path

The serial implementation resides primarily in `src/Topology.cc`:

1. `SplitMultiNodes()` admits degree-three candidates only through the shared BCC classifier and the eligibility guards above.
2. The existing node/neighbor backup and temporary `SplitNode()` workflow evaluates the single meaningful glide-arm separation.
3. The degree-two trial velocity, split-power threshold, and saved trial positions apply only to the binary-junction case; behavior for nodes with four or more arms remains on the established path.

4. After the final split succeeds, whichever result has degree two is marked `CORNER_NODE` and its complete constraint value is queued for remote domains when the operation is global.

Temporary arm storage was enlarged to hold both selected glide arms for a three-arm split, and temporary reposition flags are initialized before each evaluation. These details avoid under-allocation and stale-state behavior when the new, smaller topology enters code originally sized for nodes of degree four or greater.

5 Parallel topology path

The parallel selector continues to use the staged `SplitMultiNodesParallel()` workflow:

1. `BuildGhostMultiNodeList()` and `BuildLocalMultiNodeList()` call the same splittability predicate for native and ghost degree-three nodes.
2. `SMNEvalSplitForces()` constructs the eligible trial and computes each domain’s partial segment forces.
3. The existing force-buffer exchange reduces those contributions onto the owning domain.
4. `SMNEvalMultiNodeSplits()` performs the final mobility, separation, power, and direction tests and executes the winning split.

All three degree-three split-info slots remain present in the communication layout, but the two arm sets containing the junction arm are skipped. Keeping the full slot indexing makes force packing and unpacking consistent across domains. The parallel evaluation mirrors the serial fixes: adequate arm and force storage, combined mobility status, guarded velocity normalization, zero degree-two velocity, the $P_s > 1$ threshold, saved final positions, and corner marking after a successful split.

In a non-PARALLEL build, initialization forces `useParallelSplitMultiNode` to zero. Thus the parallel selector is only meaningful when ParaDiS was compiled with parallel topology support.

6 Corner preservation and remeshing

`CORNER_NODE` is the constraint bit `0x00000010` (decimal 16). It identifies the physical degree-two corner created by a successful binary junction split. The bit is included in the valid constraint mask. During topology initialization and native/ghost candidate-list construction, stale corner markers are removed from nodes observed with a degree other than two.

Both remesh coarsening paths apply the same length rule. An edge touching a corner may be merged only when

$$L \leq 2r_{\text{ann}}. \quad (7)$$

The candidate phase avoids selecting a corner whose two attached edges are both longer than that limit. The execution phase gates the two neighboring edges independently, so arbitrary arm ordering cannot cause a long corner-attached edge to collapse merely because the other edge is short. Very short pathological segments may still be removed.

7 Cross-domain constraint propagation

ParaDiS topology operations are replayed on neighboring MPI domains before the immediate remesh stage. The new append-only operation `REM_OP_RESET_CONSTRAINT` carries a node tag and the complete integer constraint value:

1. `SplitNode()` first emits its existing create/connectivity operations.
2. The successful serial or parallel caller marks the degree-two node and appends `AddOpResetConstraint()`.

3. `FixRemesh()` replays the byte stream in order, so a newly created ghost exists before its constraint is reset.

Appending the enumeration value preserves the numeric identifiers of all pre-existing remote operations. Nevertheless, the packed operation stream is an executable-level protocol: every MPI rank must use the same rebuilt binary.

8 Control parameters

Parameter	Meaning
<code>split3node</code>	Boolean switch for eligible BCC three-arm binary-junction splitting. The default is 1 (enabled), and input is normalized to zero or one.
<code>useParallelSplitMultiNode</code>	Selects the topology implementation: 0 uses <code>SplitMultiNodes()</code> , while 1 uses <code>SplitMultiNodesParallel()</code> in a <code>PARALLEL</code> build.
<code>splitMultiNodeFreq</code> <code>rann</code>	Existing cadence controlling which cycles evaluate multi-node splits. Existing annihilation distance; the new split and corner-remesh thresholds are defined from $2r_{\text{ann}}$.

No new material type or mobility law is introduced. The feature activates only when the existing material selection is BCC.

9 Implementation map

Files	Responsibility
<code>include/Node.h</code> , <code>src/Initialize.cc</code>	Define and validate <code>CORNER_NODE</code> .
<code>include/Param.h</code> , <code>src/Param.cc</code> , <code>src/InputSanity.cc</code> , <code>src/ParadisInit.cc</code>	Bind, normalize, and report <code>split3node</code> .
<code>include/Topology.h</code> , <code>src/Topology.cc</code>	Shared classifier and serial split implementation.
<code>src/SMNSupport.cc</code> , <code>src/SplitMultiNodesParallel.cc</code>	Parallel candidate lists, arm sets, and orchestration.
<code>src/SMNEvalSplitForces.cc</code> , <code>src/SMNEvalMultiNodeSplits.cc</code>	Distributed trial-force evaluation and owning-domain split selection.
<code>src/Remesh.cc</code>	Corner-aware candidate and per-edge coarsening gates.
<code>include/OpList.h</code> , <code>src/RemoteOps.cc</code> , <code>src/FixRemesh.cc</code>	Serialize, emit, and replay full constraint resets.
<code>tests/bcc_binary_junction_node.</code> <code>ctrl</code> , <code>tests/bcc_binary_junction.</code> <code>node.data</code>	One-step regression control and four-node input network.

10 Regression fixture

10.1 Input geometry

The data file contains four nodes in a periodic box. The central node has one $\langle 100 \rangle$ junction arm of magnitude approximately 1.1547 and two $\frac{1}{2}\langle 111 \rangle$ glide arms of unit magnitude. Their line directions make the junction non-planar under the five-degree classifier. The three endpoint nodes are pinned so the one-step test isolates the topology operation from endpoint motion.

The control file selects `BCC_0`, enables `split3node`, evaluates multi-node splitting every cycle, applies a zz stress of 5×10^8 , and runs one step. It defaults to the serial selector.

10.2 Commands

Run from the repository's `tests` directory:

```
../bin/paradis -d bcc_binary_junction_node.data \  
    bcc_binary_junction_node.ctrl
```

Set `useParallelSplitMultiNode = 1` in the control file to exercise the parallel topology selector. For the MPI ownership/remote-operation check, use a `PARALLEL` build and run:

```
mpirun -n 2 ../bin/paradis -doms 2 1 1 \  
    -d bcc_binary_junction_node.data \  
    bcc_binary_junction_node.ctrl
```

10.3 Expected result

After one step, the restart under `bcc_binary_junction_node_results/restart/restart.data` must contain:

- five nodes in total;
- the three original pinned degree-one endpoints;
- exactly one degree-two node with constraint 16 (`CORNER_NODE`); and
- the surviving degree-three node without the corner constraint.

The serial and parallel one-rank paths should produce the same restart topology. The two-rank run additionally checks that the degree-two corner constraint survives ownership and ghost replay before remeshing.

10.4 Validation performed

The final regression review exercised both complete build configurations and all relevant topology/ownership combinations:

- serial and `PARALLEL` builds completed successfully;
- one-rank runs with topology selectors 0 and 1 produced byte-identical restart data;
- two-rank MPI runs completed with both topology selectors; and
- the final restart contained the five-node topology and unique degree-two constraint-16 corner described above.

11 Compatibility boundaries

This implementation deliberately does not generalize three-arm splitting. It applies only to BCC nodes matching the exact binary-junction signature and rejects planar, pinned, surface, degenerate, and short-segment candidates. Nodes with four or more arms retain their prior multi-node split behavior. Because `CORNER_NODE` is meaningful only for degree-two nodes, topology list construction removes the bit from any node whose degree later changes.